“‘latex

PROJECT DAWET

JavaScript Fundamentals

Minggu 8 – Assignment 2

Learning Focus

Memahami dasar-dasar pemrograman JavaScript, mulai dari variabel, tipe data, operator, percabangan, perulangan, fungsi, array, hingga object.

Asynchronous Learning Program

Learn. Build. Understand.

Pendahuluan

Pada materi sebelumnya, kita telah mempelajari HTML untuk membangun struktur halaman web dan CSS untuk mengatur tampilan serta tata letaknya.

Namun, sebuah website tidak selalu cukup hanya memiliki struktur dan tampilan. Dalam banyak kasus, website juga membutuhkan kemampuan untuk merespons tindakan pengguna, memproses data, dan menjalankan logika tertentu.

Contohnya:

- Menghitung total harga produk.
- Menampilkan pesan ketika tombol ditekan.
- Memeriksa apakah formulir sudah diisi.
- Menampilkan daftar produk berdasarkan data.
- Menghitung jumlah barang yang dipilih pengguna.

Untuk membuat perilaku tersebut, kita membutuhkan JavaScript.

JavaScript merupakan bahasa pemrograman yang banyak digunakan dalam pengembangan web. Pada sisi browser, JavaScript dapat digunakan untuk membuat halaman web menjadi interaktif.

Dalam rangkaian materi Assignment 2, JavaScript juga akan menjadi dasar untuk mempelajari manipulasi DOM, pengolahan formulir, serta komunikasi dengan backend.

Tujuan Pembelajaran

Setelah menyelesaikan materi ini, peserta diharapkan mampu:

1. Menjelaskan fungsi JavaScript dalam pengembangan web.
2. Menghubungkan file JavaScript dengan HTML.
3. Menggunakan variabel dan tipe data.
4. Menggunakan operator dalam ekspresi.
5. Membuat percabangan berdasarkan kondisi.
6. Menggunakan perulangan untuk memproses data.
7. Membuat dan memanggil fungsi.
8. Menggunakan array dan object sederhana.
9. Menyelesaikan permasalahan sederhana menggunakan logika JavaScript.

Mengenal JavaScript

Apa Itu JavaScript?

JavaScript adalah bahasa pemrograman yang digunakan untuk membuat aplikasi interaktif.

Dalam pengembangan web, JavaScript dapat berjalan di browser pengguna dan berinteraksi dengan halaman HTML melalui Document Object Model (DOM).

Contoh penerapannya:

- Tombol yang menjalankan suatu aksi.
- Formulir yang memeriksa input pengguna.
- Menu navigasi interaktif.
- Perhitungan harga secara otomatis.
- Pengambilan dan penampilan data dari server.

JavaScript juga dapat digunakan di luar browser, misalnya melalui Node.js untuk membangun aplikasi backend.

Namun, pada materi ini kita akan memulai dari penggunaan JavaScript di browser.

Hubungan HTML, CSS, dan JavaScript

Ketiganya memiliki peran yang berbeda, tetapi saling melengkapi dalam membangun sebuah website.

Teknologi	Peran Utama
HTML	Membentuk struktur dan konten halaman.
CSS	Mengatur tampilan, warna, ukuran, dan tata letak.
JavaScript	Menambahkan logika dan perilaku interaktif.

Sebagai contoh, sebuah tombol pada website dapat dibangun menggunakan HTML, diberi warna menggunakan CSS, kemudian diberi fungsi tertentu menggunakan JavaScript.

Cara Menambahkan JavaScript ke HTML

Terdapat beberapa cara untuk menggunakan JavaScript dalam dokumen HTML.

Inline JavaScript

JavaScript dapat ditulis langsung pada atribut HTML tertentu.

Contoh:

```
<button onclick="alert('Halo DAWET!')">
    Klik Saya
</button>
```

Cara ini mudah digunakan untuk demonstrasi sederhana, tetapi tidak disarankan untuk pengembangan proyek yang lebih terstruktur karena logika JavaScript tercampur dengan HTML.

Internal JavaScript

JavaScript dapat ditulis menggunakan elemen `<script>` di dalam file HTML.

Contoh:

```
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <title>Belajar JavaScript</title>
</head>
<body>

    <h1>Belajar JavaScript</h1>

    <script>
        console.log("Halo DAWET!");
    </script>
```

```
</body>
</html>
```

External JavaScript

Cara yang direkomendasikan untuk proyek yang memiliki banyak kode JavaScript adalah memisahkan kode ke dalam file tersendiri.

Contoh struktur proyek:

```
belajar-javascript/
|
|-- index.html
|-- script.js
```

Isi file `index.html`:

```
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <meta name="viewport"
          content="width=device-width, initial-scale=1.0">
    <title>Belajar JavaScript</title>
</head>
<body>

    <h1>Belajar JavaScript</h1>

    <script src="script.js"></script>
</body>
</html>
```

Isi file `script.js`:

```
console.log("Halo DAWET!");
```

Atribut `src` digunakan untuk menentukan lokasi file JavaScript yang ingin dihubungkan.

Pada contoh tersebut, file JavaScript berada dalam direktori yang sama dengan file HTML.

Untuk proyek ini, kita akan menggunakan external JavaScript agar kode lebih terorganisasi.

Mengenal Console

Console merupakan fitur pada browser yang dapat digunakan untuk menampilkan informasi dan membantu proses debugging.

Salah satu perintah yang sering digunakan adalah:

```
console.log("Halo, dunia!");
```

Untuk membuka console di browser:

1. Buka halaman HTML melalui browser.
2. Tekan tombol F12 atau klik kanan lalu pilih *Inspect*.
3. Pilih tab *Console*.
4. Perhatikan keluaran dari perintah `console.log()`.

Console akan menjadi alat penting selama proses belajar JavaScript.

Variabel dan Tipe Data

Apa Itu Variabel?

Variabel merupakan tempat untuk menyimpan nilai yang dapat digunakan kembali dalam program.

Contohnya, sebuah website UMKM mungkin perlu menyimpan informasi nama produk, harga, dan jumlah stok.

Dalam JavaScript, variabel dapat dibuat menggunakan kata kunci `let`, `const`, atau `var`.

Deklarasi Variabel

1. let

Digunakan untuk membuat variabel yang nilainya dapat diubah.

```
let namaProduk = "Kopi Lokal";

namaProduk = "Kopi Susu";

console.log(namaProduk);
```

Hasil:

```
Kopi Susu
```

2. const

Digunakan untuk membuat variabel yang tidak dapat diisi ulang melalui assignment setelah deklarasi.

```
const namaToko = "DAWET Store";

console.log(namaToko);
```

Contoh berikut akan menghasilkan error:

```
const harga = 15000;
harga = 20000;
```

Perlu diketahui bahwa `const` tidak selalu berarti seluruh isi objek atau array menjadi tidak dapat diubah. Yang tidak dapat diubah adalah pengikatan variabelnya ke nilai lain.

3. var

`var` merupakan cara deklarasi variabel yang digunakan pada JavaScript versi lama.

```
var nama = "Budi";
console.log(nama);
```

Pada pembelajaran ini, utamakan penggunaan `let` dan `const` karena keduanya memiliki aturan cakupan variabel yang lebih mudah dikelola untuk kode modern.

Aturan Penamaan Variabel

Beberapa aturan dasar penamaan variabel:

- Nama variabel dapat menggunakan huruf, angka, garis bawah, dan tanda dolar.
- Nama variabel tidak boleh diawali angka.
- Nama variabel tidak boleh menggunakan kata kunci JavaScript.
- JavaScript membedakan huruf besar dan kecil.

Contoh:

```
let namaProduk = "Nasi Goreng";
let hargaProduk = 15000;
let jumlahStok = 20;
```

Gunakan nama yang menjelaskan isi variabel agar kode lebih mudah dibaca.

Tipe Data

Tipe data menjelaskan jenis nilai yang disimpan dalam sebuah variabel.

Beberapa tipe data dasar JavaScript adalah:

1. String

Digunakan untuk menyimpan teks.

```
let nama = "Kopi Susu";
let kategori = 'Minuman';
```

2. Number

Digunakan untuk menyimpan angka, baik bilangan bulat maupun desimal.

```
let harga = 15000;
let rating = 4.8;
```

3. Boolean

Digunakan untuk menyimpan nilai logika `true` atau `false`.

```
let tersedia = true;
let sudahDibayar = false;
```

4. Undefined

Menunjukkan variabel yang telah dideklarasikan, tetapi belum diberi nilai.

```
let alamat;

console.log(alamat);
```

5. Null

Digunakan untuk menyatakan bahwa suatu nilai sengaja dikosongkan.

```
let nomorTelepon = null;
```

6. Object

Digunakan untuk mengelompokkan data dalam bentuk pasangan properti dan nilai.

```
const produk = {
    nama: "Kopi Susu",
    harga: 15000,
    tersedia: true
};
```

7. Array

Digunakan untuk menyimpan kumpulan nilai dalam satu variabel.

```
const daftarProduk = [
    "Kopi Susu",
    "Teh Manis",
    "Roti Bakar"
];
```

Memeriksa Tipe Data

Gunakan operator `typeof` untuk memeriksa tipe data suatu nilai.

```
let nama = "DAWET";
let harga = 15000;
let aktif = true;

console.log(typeof nama);
console.log(typeof harga);
console.log(typeof aktif);
```

Hasil:

```
string
number
boolean
```

Perlu diperhatikan bahwa `typeof null` menghasilkan `"object"` karena perilaku historis JavaScript.

Operator dalam JavaScript

Operator digunakan untuk melakukan operasi terhadap nilai atau variabel.

Operator Aritmatika

Operator aritmatika digunakan untuk melakukan perhitungan matematika.

Operator	Fungsi
+	Penjumlahan
-	Pengurangan
*	Perkalian
/	Pembagian
%	Sisa hasil bagi atau modulo
**	Perpangkatan

Contoh:

```
let harga = 15000;
let jumlah = 3;

let total = harga * jumlah;

console.log(total);
```

Hasil:

```
45000
```

Operator Perbandingan

Operator perbandingan digunakan untuk membandingkan dua nilai dan menghasilkan nilai Boolean.

Operator	Fungsi
==	Membandingkan nilai dengan konversi tipe tertentu.
===	Membandingkan nilai dan tipe data.
!=	Membandingkan ketidaksamaan

Operator	Fungsi
	nilai.
!==	Membandingkan ketidaksamaan nilai atau tipe.
>	Lebih besar dari.
<	Lebih kecil dari.
>=	Lebih besar atau sama dengan.
<=	Lebih kecil atau sama dengan.

Contoh:

```
let harga = 15000;

console.log(harga > 10000);
console.log(harga === 15000);
```

Hasil:

```
true
true
```

Dalam praktik sehari-hari, gunakan === dan !== sebagai pilihan utama agar perbandingan lebih ketat dan mudah diprediksi.

Operator Logika

Operator logika digunakan untuk menggabungkan atau membalikkan kondisi.

Operator	Fungsi
&&	AND: semua kondisi harus bernilai benar.
||	OR: minimal satu kondisi harus benar.
!	NOT: membalikkan nilai logika.

Contoh AND:

```
let stok = 10;
let aktif = true;

console.log(stok > 0 && aktif);
```

Hasil:

```
true
```

Contoh OR:

```
let metodePembayaran = "transfer";

console.log(
    metodePembayaran === "transfer" ||
    metodePembayaran === "tunai"
);
```

Hasil:

```
true
```

Contoh NOT:

```
let tersedia = true;

console.log(!tersedia);
```

Hasil:

```
false
```

Percabangan

Apa Itu Percabangan?

Percabangan adalah mekanisme yang memungkinkan program mengambil keputusan berdasarkan kondisi tertentu.

Sebagai contoh, sebuah sistem UMKM dapat menentukan apakah suatu produk masih tersedia berdasarkan jumlah stoknya.

if

Struktur `if` menjalankan kode ketika kondisi bernilai benar.

```
let stok = 10;

if (stok > 0) {
    console.log("Produk tersedia");
}
```

Karena stok lebih besar dari nol, pesan `Produk tersedia` akan ditampilkan.

if dan else

Tambahkan `else` jika program perlu menjalankan alternatif ketika kondisi salah.

```
let stok = 0;

if (stok > 0) {
    console.log("Produk tersedia");
```

```
} else {
    console.log("Produk habis");
}
```

if, else if, dan else

Struktur ini digunakan ketika terdapat beberapa kemungkinan kondisi.

```
let totalBelanja = 75000;

if (totalBelanja >= 100000) {
    console.log("Diskon 20%");
} else if (totalBelanja >= 50000) {
    console.log("Diskon 10%");
} else {
    console.log("Tidak ada diskon");
}
```

Program akan memeriksa kondisi dari atas ke bawah. Ketika suatu kondisi terpenuhi, blok yang sesuai dijalankan dan rangkaian percabangan tersebut selesai.

switch

`switch` dapat digunakan ketika kita ingin memeriksa satu nilai terhadap beberapa kemungkinan nilai yang berbeda.

```
let kategori = "minuman";

switch (kategori) {
    case "makanan":
        console.log("Kategori makanan");
        break;

    case "minuman":
        console.log("Kategori minuman");
        break;

    case "camilan":
        console.log("Kategori camilan");
        break;

    default:
        console.log("Kategori tidak dikenal");
}
```

Perintah `break` digunakan untuk menghentikan eksekusi pada case yang telah cocok agar program tidak melanjutkan ke case berikutnya.

`default` dijalankan jika tidak ada case yang cocok.

Perulangan

Apa Itu Perulangan?

Perulangan atau loop digunakan untuk menjalankan blok kode berulang kali.

Misalnya, kita ingin menampilkan daftar sepuluh produk. Daripada menulis perintah yang sama sepuluh kali, kita dapat menggunakan perulangan.

for

Perulangan `for` biasanya digunakan ketika jumlah pengulangan diketahui atau dapat ditentukan.

Struktur dasarnya:

```
for (inisialisasi; kondisi; perubahan) {
    // kode yang dijalankan
}
```

Contoh:

```
for (let i = 1; i <= 5; i++) {
    console.log("Produk ke-" + i);
}
```

Penjelasan:

- `let i = 1`: nilai awal penghitung.
- `i <= 5`: perulangan berjalan selama kondisi benar.
- `i++`: menaikkan nilai penghitung sebanyak satu pada setiap putaran.

while

Perulangan `while` menjalankan kode selama kondisi bernilai benar.

```
let stok = 3;

while (stok > 0) {
    console.log("Stok tersisa: " + stok);
    stok--;
}
```

Perulangan berhenti ketika nilai `stok` tidak lagi lebih besar dari nol.

Pastikan kondisi perulangan dapat berubah sehingga pada akhirnya menjadi salah.

Jika kondisi selalu benar dan tidak ada mekanisme penghentian, program dapat mengalami infinite loop, yaitu perulangan yang tidak pernah selesai.

break dan continue

`break` digunakan untuk menghentikan perulangan secara langsung.

```
for (let i = 1; i <= 5; i++) {
    if (i === 3) {
        break;
    }

    console.log(i);
}
```

Hasil:

```
1
2
```

Sementara itu, `continue` digunakan untuk melewati satu putaran dan melanjutkan ke putaran berikutnya.

```
for (let i = 1; i <= 5; i++) {
    if (i === 3) {
        continue;
    }

    console.log(i);
}
```

Hasil:

```
1
2
4
5
```

Fungsi

Apa Itu Fungsi?

Fungsi adalah blok kode yang dirancang untuk menjalankan tugas tertentu dan dapat dipanggil kembali ketika dibutuhkan.

Fungsi membantu kita menghindari penulisan kode yang sama secara berulang.

Sebagai contoh, jika kita sering menghitung total harga produk, kita dapat membuat satu fungsi untuk menangani perhitungan tersebut.

Membuat Function Declaration

Contoh:

```
function sapa() {
    console.log("Halo, selamat datang!");
}
```

Kode di atas hanya mendefinisikan fungsi. Agar dijalankan, kita perlu memanggilnya:

```
sapa();
```

Parameter dan Argument

Fungsi dapat menerima data melalui parameter.

```
function sapa(nama) {
    console.log("Halo, " + nama);
}

sapa("Budi");
```

Pada contoh tersebut:

- `nama` adalah parameter.
- `"Budi"` adalah argument yang dikirim ketika fungsi dipanggil.

return

Fungsi dapat menghasilkan nilai menggunakan perintah `return`.

```
function hitungTotal(harga, jumlah) {
    return harga * jumlah;
}

let total = hitungTotal(15000, 3);

console.log(total);
```

Hasil:

```
45000
```

Perbedaan penting:

- `console.log()` menampilkan informasi pada console.
- `return` mengembalikan nilai dari fungsi kepada bagian program yang memanggilnya.

Function Expression

Fungsi juga dapat disimpan ke dalam variabel.

```
const hitungLuas = function(panjang, lebar) {
    return panjang * lebar;
```

```
};

console.log(hitungLuas(5, 4));
```

Arrow Function

JavaScript modern menyediakan bentuk penulisan fungsi yang disebut arrow function.

```
const hitungTotal = (harga, jumlah) => {
    return harga * jumlah;
};

console.log(hitungTotal(15000, 3));
```

Untuk fungsi dengan satu ekspresi sederhana, penulisannya dapat dibuat lebih singkat:

```
const tambah = (a, b) => a + b;

console.log(tambah(5, 3));
```

Pada tahap awal, peserta cukup memahami cara mendefinisikan fungsi, memberikan parameter, memanggil fungsi, dan mengembalikan nilai.

Array

Apa Itu Array?

Array adalah struktur data yang digunakan untuk menyimpan kumpulan nilai dalam satu variabel.

Contoh:

```
const produk = [
    "Kopi Susu",
    "Teh Manis",
    "Roti Bakar"
];
```

Setiap nilai di dalam array memiliki posisi yang disebut index.

Index array JavaScript dimulai dari angka nol.

Index	0	1	2
Nilai	Kopi Susu	Teh Manis	Roti Bakar

Mengakses Elemen Array

Gunakan index untuk mengambil elemen tertentu.

```
const produk = [
    "Kopi Susu",
    "Teh Manis",
    "Roti Bakar"
];

console.log(produk[0]);
console.log(produk[1]);
```

Hasil:

```
Kopi Susu
Teh Manis
```

Mengubah Elemen Array

Elemen array dapat diubah melalui index.

```
const produk = [
    "Kopi Susu",
    "Teh Manis"
];

produk[1] = "Es Teh";

console.log(produk);
```

Meskipun menggunakan `const`, elemen array tetap dapat diubah. Yang tidak dapat diubah adalah pengikatan variabel `produk` ke array lain.

Properti length

Properti `length` digunakan untuk mengetahui jumlah elemen dalam array.

```
const produk = [
    "Kopi Susu",
    "Teh Manis",
    "Roti Bakar"
];

console.log(produk.length);
```

Hasil:

```
3
```

Method Dasar Array

Beberapa method yang umum digunakan:

Method	Fungsi
push()	Menambahkan elemen di akhir array.
pop()	Menghapus elemen terakhir.
unshift()	Menambahkan elemen di awal array.
shift()	Menghapus elemen pertama.
includes()	Memeriksa apakah nilai terdapat dalam array.
indexOf()	Mencari posisi pertama suatu nilai.

Contoh:

```
const produk = ["Kopi", "Teh"];

produk.push("Roti");

console.log(produk);
```

Hasil:

```
["Kopi", "Teh", "Roti"]
```

Menggunakan Perulangan pada Array

Array dapat diproses menggunakan perulangan.

```
const produk = [
    "Kopi Susu",
    "Teh Manis",
    "Roti Bakar"
];

for (let i = 0; i < produk.length; i++) {
    console.log(produk[i]);
}
```

Kita juga dapat menggunakan `for...of` untuk membaca setiap nilai array.

```
for (const namaProduk of produk) {
    console.log(namaProduk);
}
```

Object

Apa Itu Object?

Object merupakan struktur data yang digunakan untuk menyimpan kumpulan informasi dalam bentuk pasangan key dan value.

Sebagai contoh, sebuah produk UMKM memiliki nama, harga, kategori, dan status ketersediaan.

Informasi tersebut dapat dikelompokkan dalam satu object.

```
const produk = {
    nama: "Kopi Susu",
    harga: 15000,
    kategori: "Minuman",
    tersedia: true
};
```

Pada contoh tersebut:

- `nama` adalah key dengan nilai `"Kopi Susu"`.
- `harga` adalah key dengan nilai `15000`.
- `kategori` adalah key dengan nilai `"Minuman"`.
- `tersedia` adalah key dengan nilai `true`.

Mengakses Properti Object

Terdapat dua cara umum untuk mengakses properti object.

Menggunakan dot notation:

```
console.log(produk.nama);
console.log(produk.harga);
```

Menggunakan bracket notation:

```
console.log(produk["kategori"]);
```

Mengubah Properti Object

Properti object dapat diubah atau ditambahkan.

```
const produk = {
    nama: "Kopi Susu",
    harga: 15000
};

produk.harga = 18000;
```

```
produk.stok = 20;

console.log(produk);
```

Array of Objects

Dalam aplikasi nyata, data biasanya tidak hanya terdiri dari satu object.

Kita dapat menggabungkan array dan object untuk menyimpan banyak data produk.

```
const daftarProduk = [
    {
        nama: "Kopi Susu",
        harga: 15000
    },
    {
        nama: "Teh Manis",
        harga: 8000
    },
    {
        nama: "Roti Bakar",
        harga: 12000
    }
];
```

Untuk mengakses nama produk pertama:

```
console.log(daftarProduk[0].nama);
```

Untuk menampilkan seluruh produk:

```
for (const produk of daftarProduk) {
    console.log(produk.nama);
}
```

Konsep array of objects akan sering digunakan ketika kita mulai mengolah data produk pada proyek website UMKM.

Praktik Terpadu

Pada bagian ini, kita akan menggabungkan beberapa konsep yang telah dipelajari ke dalam satu program sederhana.

Studi Kasus: Sistem Perhitungan Belanja

Buat sebuah file bernama `script.js`.

Program akan menyimpan beberapa data produk, menghitung total harga, dan memberikan informasi berdasarkan jumlah belanja.

```
const daftarProduk = [
    {
        nama: "Kopi Susu",
        harga: 15000,
        jumlah: 2
    },
    {
        nama: "Roti Bakar",
        harga: 12000,
        jumlah: 1
    },
    {
        nama: "Teh Manis",
        harga: 8000,
        jumlah: 3
    }
];

function hitungTotal(harga, jumlah) {
    return harga * jumlah;
}

let totalBelanja = 0;

console.log("DAFTAR BELANJA");

for (const produk of daftarProduk) {
    const subtotal = hitungTotal(
        produk.harga,
        produk.jumlah
    );

    totalBelanja += subtotal;

    console.log(
        produk.nama + ": Rp" + subtotal
    );
}

console.log("----------------");

console.log(
    "Total: Rp" + totalBelanja
);

if (totalBelanja >= 50000) {
    console.log("Selamat, Anda mendapat diskon!");
} else {
```

```
    console.log("Belum memenuhi syarat diskon.");
}
```

Analisis Program

Program tersebut menggabungkan beberapa konsep:

1. **Array of Objects**: menyimpan beberapa data produk.
2. **Function**: menghitung subtotal berdasarkan harga dan jumlah.
3. **Perulangan**: memproses setiap produk secara berurutan.
4. **Variabel**: menyimpan total belanja.
5. **Operator**: melakukan perhitungan.
6. **Percabangan**: menentukan apakah pelanggan memperoleh diskon.

Jalankan program melalui browser dengan menghubungkan file JavaScript ke HTML, kemudian buka tab Console untuk melihat hasilnya.

Latihan Mandiri

Kerjakan latihan berikut untuk menguji pemahaman terhadap materi.

Latihan 1: Variabel dan Operator

Buat variabel berikut:

- Nama toko.
- Nama produk.
- Harga produk.
- Jumlah produk.

Hitung total harga menggunakan operator aritmatika dan tampilkan hasilnya menggunakan `console.log()`.

Latihan 2: Percabangan

Buat variabel `stok` dengan nilai tertentu.

Gunakan percabangan untuk menentukan pesan:

- Jika stok lebih dari nol, tampilkan `Produk tersedia`.
- Jika stok sama dengan nol, tampilkan `Produk habis`.

- Jika stok kurang dari nol, tampilkan `Data stok tidak valid`.

Latihan 3: Perulangan

Buat array yang berisi minimal lima nama produk.

Gunakan perulangan untuk menampilkan seluruh nama produk ke console.

Latihan 4: Fungsi

Buat fungsi bernama `hitungDiskon()`.

Fungsi menerima dua parameter:

- Harga awal.
- Persentase diskon.

Fungsi harus mengembalikan harga setelah diskon.

Contoh:

```
hitungDiskon(100000, 10);
```

Hasil yang diharapkan:

```
90000
```

Latihan 5: Object dan Array

Buat array bernama `daftarProduk` yang berisi minimal tiga object produk.

Setiap object harus memiliki:

- nama
- harga
- stok

Kemudian:

1. Tampilkan seluruh nama produk.
2. Tampilkan harga setiap produk.
3. Hitung jumlah seluruh stok.
4. Tampilkan produk yang stoknya lebih dari nol.

Evaluasi Pemahaman

Jawablah pertanyaan berikut menggunakan pemahaman sendiri. Peserta tidak cukup hanya menyalin kode dari materi.

1. Apa perbedaan penggunaan `let` dan `const`?
2. Apa perbedaan tipe data String, Number, dan Boolean?
3. Mengapa operator === lebih disarankan daripada ==?
4. Kapan kita menggunakan percabangan `if-else`?
5. Apa perbedaan perulangan `for` dan `while`?
6. Apa perbedaan parameter dan argument dalam sebuah fungsi?
7. Mengapa array menggunakan index yang dimulai dari nol?
8. Apa perbedaan array dan object?
9. Mengapa array of objects berguna untuk menyimpan data produk?
10. Jelaskan bagaimana program perhitungan belanja pada materi ini bekerja.

Kriteria Penyelesaian

Peserta dinyatakan telah menyelesaikan materi ini apabila mampu:

- Membuat file JavaScript eksternal dan menghubungkannya dengan HTML.
- Menggunakan console untuk melihat hasil program dan membantu debugging.
- Membuat serta menggunakan variabel.
- Memahami tipe data dasar.
- Menggunakan operator aritmatika, perbandingan, dan logika.
- Membuat percabangan.
- Menggunakan perulangan.
- Membuat fungsi dengan parameter dan return.
- Mengolah array dan object sederhana.
- Menyelesaikan latihan tanpa hanya menyalin contoh dari materi.

Penutup

JavaScript memberikan kemampuan kepada sebuah website untuk menjalankan logika dan merespons kebutuhan pengguna.

Pada materi ini, kita telah mempelajari fondasi pemrograman yang akan digunakan kembali dalam materi-materi berikutnya.

Variabel membantu menyimpan data, operator memproses nilai, percabangan mengambil keputusan, perulangan mengerjakan tugas berulang, sedangkan fungsi membantu menyusun kode agar lebih terstruktur.

Array dan object kemudian memberikan cara untuk mengelola kumpulan data yang lebih kompleks.

Konsep-konsep tersebut merupakan fondasi penting sebelum kita mempelajari bagaimana JavaScript berinteraksi langsung dengan elemen HTML melalui DOM.

Ingat, memahami logika lebih penting daripada sekadar menghafal sintaks.

Learn. Build. Understand.

PROJECT DAWET

Referensi Pembelajaran

Gunakan dokumentasi berikut sebagai referensi tambahan:

1. MDN Web Docs – JavaScript: https://developer.mozilla.org/en-US/docs/Web/JavaScript
2. MDN – JavaScript First Steps: https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Scripting
3. MDN – JavaScript Guide: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide
4. JavaScript.info: https://javascript.info/